



抖音稳定性优化

思路和疑难案例实战分析

张林

讲师自我介绍

张林

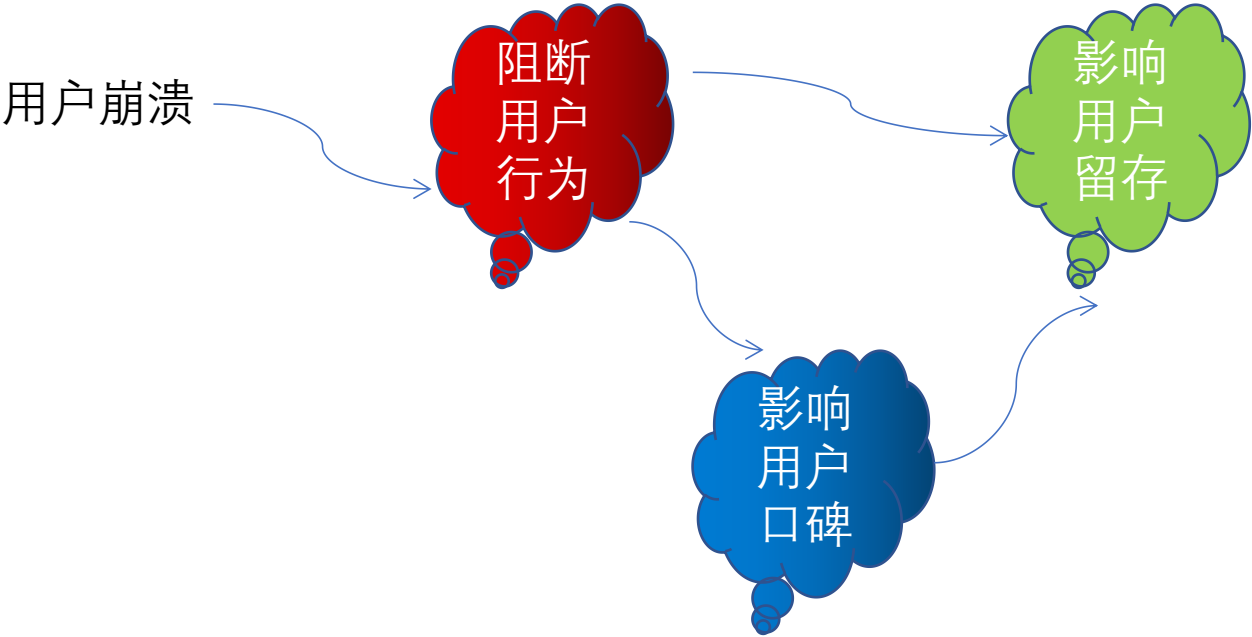
张林11年毕业，至今11年开发经验，期间做过Android应用开发、Linux逆向、跨端SDK，先后从事多个亿级应用的性能和稳定性相关工作，加入字节跳动以来，一直负责抖音相关稳定性治理和优化工作，对Android底层、Linux内核和逆向有浓厚的兴趣。

- 
- 01 背景与挑战
 - 02 目标和思路
 - 03 疑难攻坚专项
 - 04 发版质量专项
 - 05 线上容灾专项
 - 06 质量左移专项
 - 07 总结和展望

01

背景与挑战

背景-为什么要做稳定性?



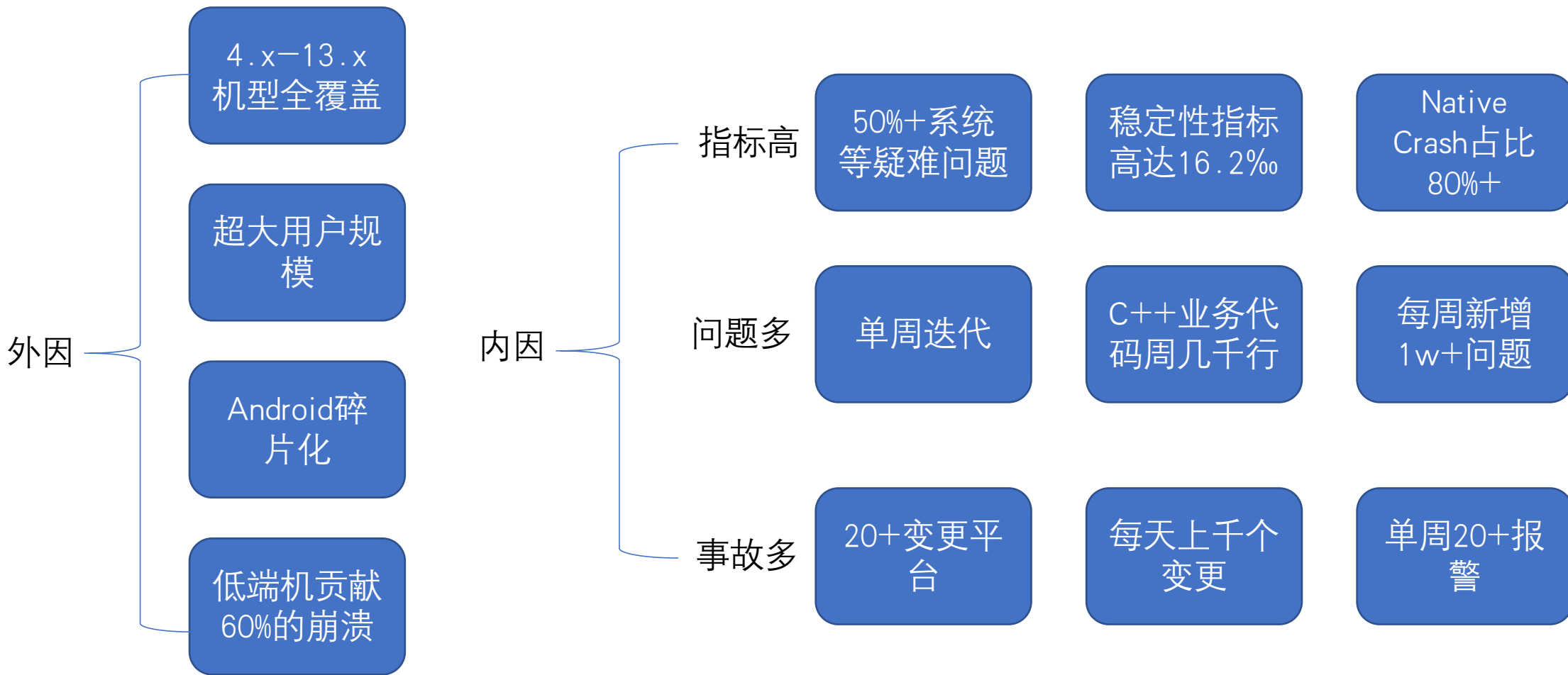
业务价值

1% Crash

折算

平均年度人均活跃天数-0.268%

挑战



02

目标和思路

目标思路

整体目标

持续提升抖音产品质量

保障业务稳健发展

指导思想

减存控增防突发

治理思路

疑难攻坚专项：减存——Top issue治理，降低水位

发版质量专项：控增——保障业务迭代，防止劣化

线上容灾专项：防突发——及时止损，减少劣化

质量左移专项：发现——更极致发现问题，降低成本

03

疑难攻坚专项

疑难攻坚-Native Crash

本次将重点讲解

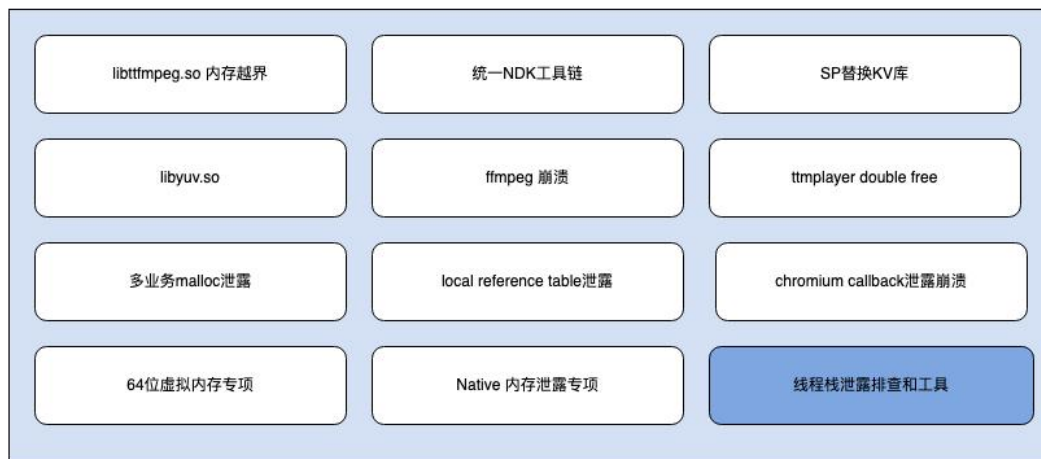
Art Thread Suspend Timeout（改指令）、dvm

LinearAlloc（改内存）

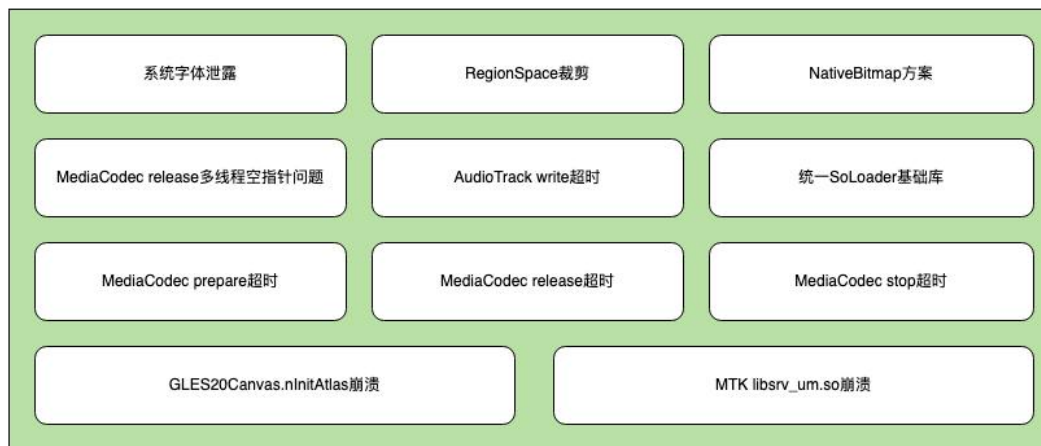
系统崩溃问题修复以及线程栈

泄露排查（Maps归类）

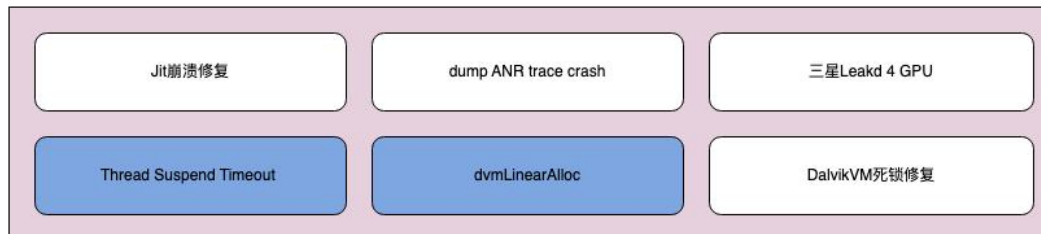
多业务问题修复



中间件问题修复



ART虚拟机相关



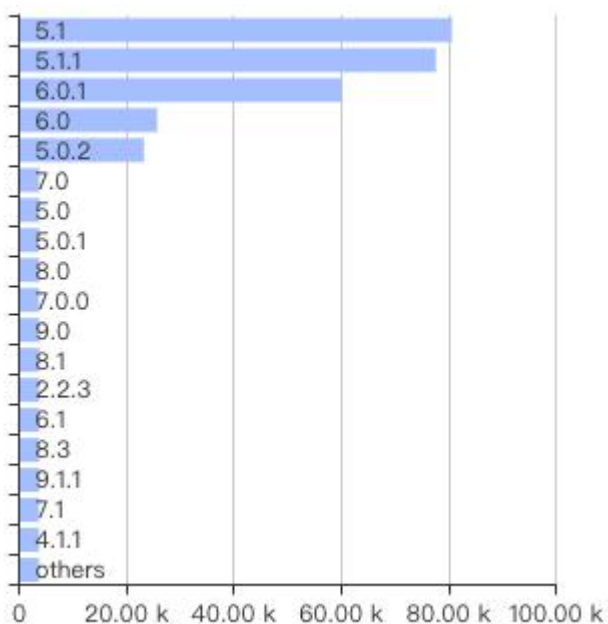
虚拟机Suspend SIGABRT

Top1 NE Crash: 每天影响10w+用户, 影响用户比例2.115‰, 总Native: 4.760‰

主堆栈

```
0  Signal 6(SIGABRT), Code -6(SI_TKILL)
1  crash message: 'art/runtime/thread_list.cc:247] Thread suspend timeout'
2  #00 pc 0003c0a0 /system/lib/libc.so (tgkill+12)
3  #01 pc 00017a01 /system/lib/libc.so (pthread_kill+52)
4  #02 pc 00018667 /system/lib/libc.so (raise+10)
5  #03 pc 00014e27 /system/lib/libc.so (__libc_android_abort+34)
6  #04 pc 0001321c /system/lib/libc.so (abort+4)
7  #05 pc 00223fa5 /system/lib/libart.so (_ZN3art7Runtime5AbortEv+160)
8  #06 pc 000a8b71 /system/lib/libart.so (_ZN3art10LogMessageD1Ev+1296)
9  #07 pc 0023efd3 /system/lib/libart.so (_ZN3artL40UnsafeLogFatalForThreadSuspendAllTimeoutEv+246)
10 #08 pc 0023f2f3 /system/lib/libart.so (_ZN3art10ThreadList10SuspendAllEv+738)
11 #09 pc 0013e6d9 /system/lib/libart.so (_ZN3art2gc4Heap30PerformHomogeneousSpaceCompactEv+1760)
12 #10 pc 0014bcef /system/lib/libart.so (_ZN3art2gc4Heap25DoPendingTransitionOrTrimEv+758)
13 #11 pc 024703df /data/dalvik-cache/arm/system@framework@boot.oat
```

OS版本



根因总结

SuspendAll时间 = ^{futex等待, 无限制} 所有线程suspend时间 + ^{>30s会abort} mutator_lock获取时间 7.0版本
STW

SuspendAll时间 = 所有线程suspend时间 + mutator_lock获取时间 <7.0版本
STW

 ^{>30s会abort}

解决方案1: 延长timeout时间

方案1: inlinehook ExclusiveLockWithTimeout 延长时间

```
bool aweExclusiveLockWithTimeout(void * lock, void * thread, long ms, int ns) {  
#endif  
    bool long_suspend = (long_suspend_ || compacting_heap_  
                          || anr_trace_dumping_ || background_gc_running_);  
    // LOGGER("long_suspend: %d", long_suspend);  
    if (artExclusiveLockWithTimeout) {  
        //LOGGER("ExclusiveLockWithTimeout: %lld", ms);  
        long suspend_ms = 10 * ms;  
        if (suspend_ms < 600 * 1000) {  
            suspend_ms = 600 * 1000;  
        }  
        while (true) {  
            if (artExclusiveLockWithTimeout(lock, thread, suspend_ms, 0)) {  
                return true;  
            } else if (!long_suspend) {  
                return false;  
            }  
        }  
    }  
    return true;  
}
```

修复后效果: 该问题平均影响用户比例从2.115‰→0.025‰, 总体Native Crash: 4.760‰→2.717‰

寻找新的解决方案

1. 还有少量的崩溃没修复，主要原因是为了解决同步的问题，在调用ExclusiveLockWithTimeout前，调用SuspendAllForDebugger
调用完之后，再调用ResumeAllForDebugger，这里的SuspendAllForDebugger本身也会有SuspendAll的操作
2. inlinehook会有包大小的增量，且可能会有稳定性问题

只能想办法看看是否有其他方案，再次看源码，发现了如下代码

```
482
483 // Block on the mutator lock until all Runnable threads release their share of access.
484 #if HAVE_TIMED_RWLOCK
485 while (true) {
486     if (Locks::mutator_lock_>ExclusiveLockWithTimeout(self, kThreadSuspendTimeoutMs, 0)) {
487         break;
488     } else if (!long_suspend_) {
489         // Reading long_suspend without the mutator lock is slightly racy, in some rare cases, this
490         // could result in a thread suspend timeout.
491         // Timeout if we wait more than kThreadSuspendTimeoutMs seconds.
492         UnsafeLogFatalForThreadSuspendAllTimeout();
493     }
494 }
495 #else
496 Locks::mutator_lock_>ExclusiveLock(self);
497 #endif
```

新方案设计

新方案

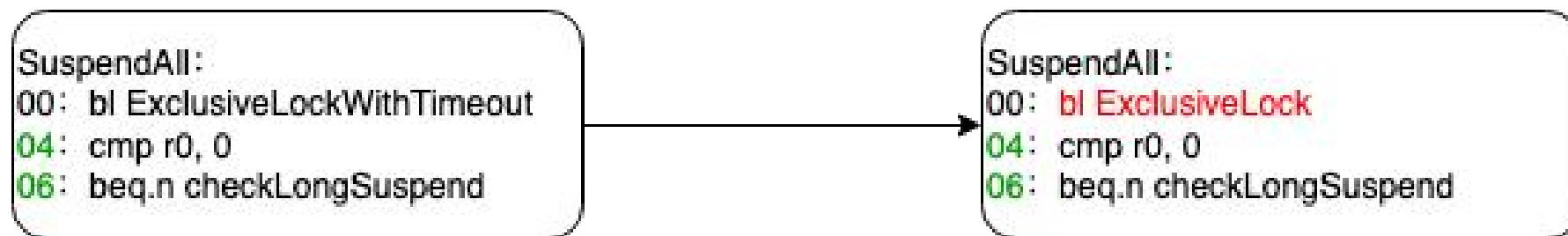


图1

ExclusiveLock VS ExclusiveLockWithTimeout:

1. 新的符号跳转范围是否OK
2. 参数不一样: `void Mutex::ExclusiveLock(Thread* self);`
`bool ReaderWriterMutex::ExclusiveLockWithTimeout(Thread* self, int64_t ms, int32_t ns)`
3. 返回值不一样: 第一个函数返回void, 第二个返回bool

新方案分析

需要处理:

1. 跳转范围: 将bl ExclusiveLockWithTimeout改成bl ExclusiveLock, 由于这两个代码都是libart.so, 符合bl指令跳转限制: $\pm 32\text{MB}$ 空间内, 可以直接替换
2. 参数问题:

```
void Mutex::ExclusiveLock(Mutex* this, Thread* self);  
bool ReaderWriterMutex::ExclusiveLockWithTimeout(ReaderWriterMutex* this,  
Thread* self, int64_t ms, int32_t ns)
```

Mutex和ReaderWriterMutex内存布局一样, 从ARM64调用约定来看, 小于7个参数用X0-X7传参, 多的从右往做依次压栈, 所以前两个参数也可以互换, 参数无需额外处理
3. **返回值问题**: ExclusiveLockWithTimeout是有bool类型的返回值的, 而ExclusiveLock是没有返回值的需要额外处理

新方案返回值处理及效果

思路1：修改多指令，并发问题，pass



思路2：用一个函数作为跳板函数，修改返回值

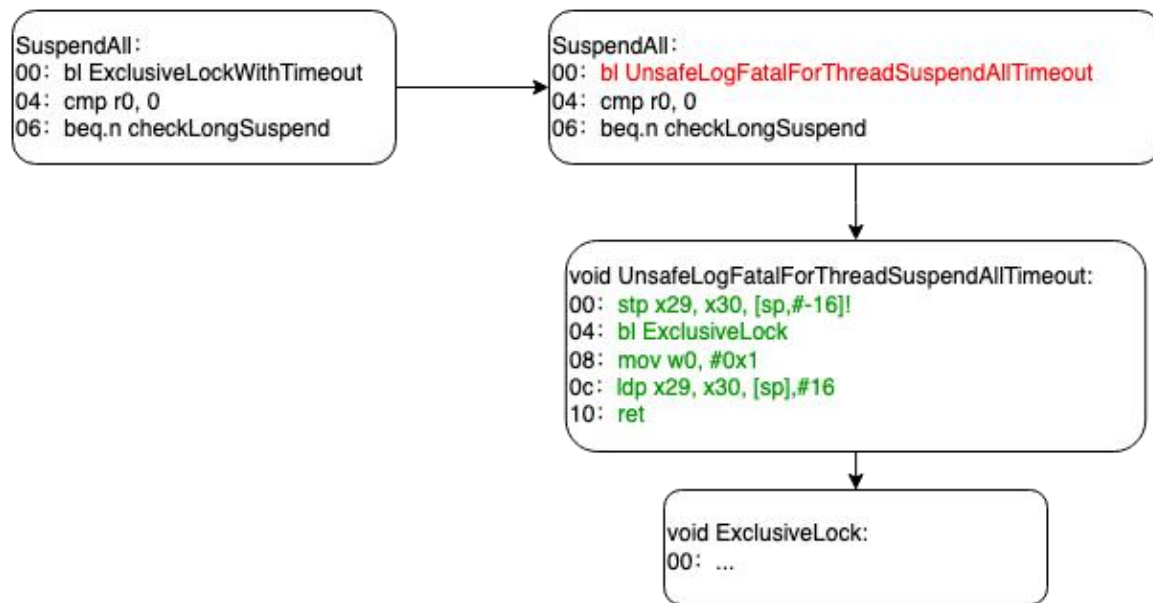


图3

新方案效果：

1. 不再调用SuspendVM, 没有负面影响
2. 不再使用inline hook, so体积减少(64位: 487k→194k, 32位: 146k→106k)
3. 完全修复

```
#21740054 SIGABRT
com.ss.android.ugc.aweme.shortvideo.GoNextFactoryFactory.goEditActivity(SourceFile:274)

dalvik.system.DexFile.defineClassNative(Native Method)
dalvik.system.DexFile.defineClass(DexFile.java:222)
dalvik.system.DexFile.loadClassBinaryName(DexFile.java:215)
dalvik.system.DexPathList.findClass(DexPathList.java:322)
dalvik.system.BaseDexClassLoader.findClass(BaseDexClassLoader.java:54)
java.lang.ClassLoader.loadClass(ClassLoader.java:497)
java.lang.ClassLoader.loadClass(ClassLoader.java:457)

com.ss.android.ugc.aweme.shortvideo.GoNextFactoryFactory.goEditActivity(SourceFile:280)
com.ss.android.ugc.aweme.shortvideo.GoNextFactoryFactory.a(SourceFile:187)
com.ss.android.ugc.aweme.shortvideo.GoNextFactoryFactory.onFinish(Unknown Source)
com.ss.android.ugc.aweme.shortvideo.GoNextFactoryFactory.onFinish(SourceFile:82)
com.ss.android.ugc.aweme.shortvideo.GoNextFactoryFactory.lambda$getMusicWaveBean$1$AVServiceImpl(SourceFile:832)
com.ss.android.ugc.aweme.shortvideo.GoNextFactoryFactory.then(Unknown Source)
bolts.Task$6.run(SourceFile:872)
android.os.Handler.handleCallback(Handler.java:733)
android.os.Handler.dispatchMessage(Handler.java:95)
android.os.Looper.loop(Looper.java:146)
android.app.ActivityThread.main(ActivityThread.java:5511)
java.lang.reflect.Method.invokeNative(Native Method)
java.lang.reflect.Method.invoke(Method.java:515)
com.android.internal.os.ZygoteInit$MethodAndArgsCaller.run(ZygoteInit.java:1283)
```

1,572

```
23:36.906 27731 20714 E FaceRecorderManager: Concat::Read Frame failed, audioCurrentTime = 0
23:37.147 27731 20714 E FaceRecorderManager: func:uninitConcatOutput, write trailer error:{0}
23:38.368 27731 20728 E TEFfmpegUtils: 1492 getNextAudioSamplesForPlayback failed
23:41.301 27731 27731 E dalvikvm: LinearAlloc exceeded capacity (16777216), last=504
23:41.301 27731 27731 E dalvikvm: VM aborting
```

原因分析

JNI_CreateJavaVM->dvmStartUp

dvmClassStartup

app启动时调用

```
gDvm.pBootLoaderAlloc = dvmLinearAllocCreate(NULL);
```

加载interface、field、method以及vtable等

dvmLinearAllocCreate

```
pHdr->mapLength = 16MB;  
...  
fd = ashmem_create_region("dalvik-  
LinearAlloc", 16MB);
```

dvmLinearAlloc

```
nextOffset = calc_from_size(size); // 块对齐  
if (nextOffset > pHdr->mapLength) {  
    dvmAbort();  
}
```

修复方案

```
576     bool    newzygoteneapAllocated;
577
578     /*
579     * TLS keys.
580     */
581     pthread_key_t pthreadKeySelf;      /* Thread*, for dvmThreadSelf */
582
583     /*
584     * Cache results of "A instanceof B".
585     */
586     AtomicCache* instanceofCache;
587
588     /* inline substitution table, used during optimization */
589     InlineSub*    inlineSubs;
590
591     /*
592     * Bootstrap class loader linear allocator.
593     */
594     LinearAllocHdr* pBootLoaderAlloc;
595
596     /*
597     * Compute some stats on loaded classes.
598     */
599     int    numLoadedClasses;
600     int    numDeclaredMethods;
601     int    numDeclaredInstFields;
602     int    numDeclaredStaticFields;
603
604     /* when using a native debugger, set this to suppress watchdog timers */
605     bool    nativeDebuggerActive;
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

核心思路：

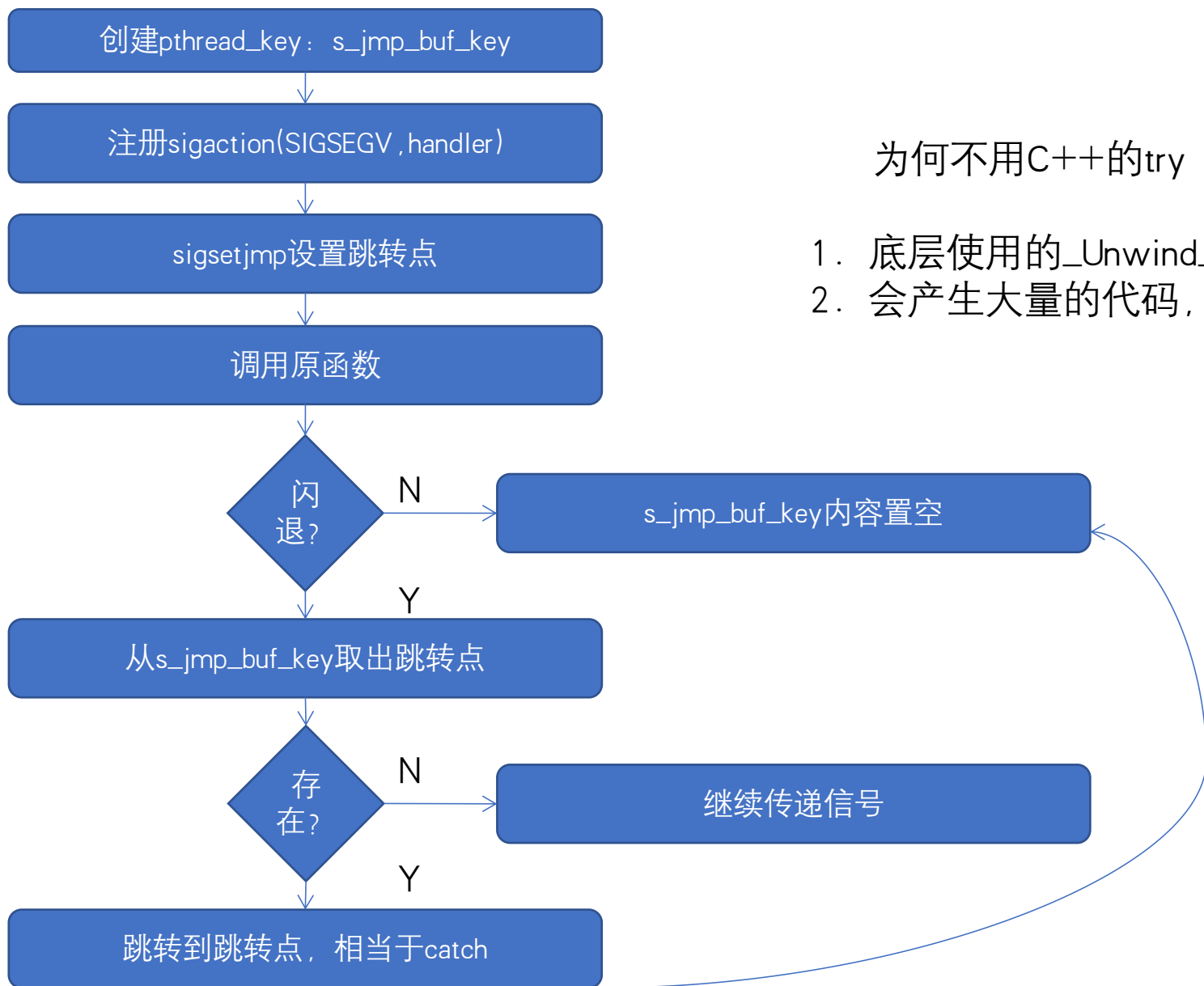
1. 以gDvm导出符号为锚点
2. 以Maps文件中LinearAlloc内核分配的第一个vma为参照物
3. 搜索gDvm内存，假设待搜索的内存是LinearAllocHdr
4. 看结构里的mapAddr是否与vma一致，且其他字段无异常
5. 更改mapAddr和mapLength

```
1 5762d000-5762e000 ---p 00000000 00:04 352953 /dev/ashmem/dalvik-LinearAlloc
   (deleted)
2 5762e000-579f8000 rw-p 00001000 00:04 352953 /dev/ashmem/dalvik-LinearAlloc
   (deleted)
3 579f8000-5832f000 rw-p 003cb000 00:04 352953 /dev/ashmem/dalvik-LinearAlloc
   (deleted)
4 5832f000-5862d000 ---p 00d02000 00:04 352953 /dev/ashmem/dalvik-LinearAlloc
   (deleted)
```

6. 会有少量多线程导致的问题，我们用try-catch框架兜底

崩溃完全修复

try-catch框架



为何不用C++的try catch

1. 底层使用的_Unwind_Resume存在符号冲突问题
2. 会产生大量的代码, 对包大小不友好

经典Case3：线程栈泄露

Native崩溃 / issue_id: 162cc9247cdd5b4ea45f908a42770e49

SIGABRT
liblog.so __android_log_assert libhwui.so libutils.so

teddy.ma,songjinshi 已处理

App小版本 := 9708 过滤

发生的次数
575.68 k

影响用户数
428.31 k

2020-02-06 20:00:00 ~ 2020-02-12 20:00:00 1d

主堆栈

```
0 pid: 25427, tid: 26990, name: RenderThread >>> com.ss.android.ugc.aweme <<<

1 signal 6 (SIGABRT), code -6 (SI_TKILL), fault addr -----

2 Abort message: 'glDeleteTextures error! GL_OUT_OF_MEMORY (0x505)'
```

3	r0	00000000	r1	0000696e	r2	00000006	r3	f6330d30
4	r4	00006353	r5	0000696e	r6	c4dff3a4	r7	0000010c
5	r8	00000000	r9	f81451e6	sl	00000505	fp	f81451b7
6	ip	f633b3c0	sp	c4dff390	lr	f62a6281	pc	f629d086
7	backtrace:							
8	#00 pc	0001d086	/system/lib/libc.so (abort+61)					
9	#01 pc	00007ee9	/system/lib/liblog.so (__android_log_assert+156)					
10	#02 pc	0007f9e5	/system/lib/libhwui.so (_ZN7android10uirenderer5debug21GlesErrorCheckWrapper14ass					
11	#03 pc	0023a3c7	/system/lib/libhwui.so (_ZN7android10uirenderer12TextureState13deleteTextureEj+98					
12	#04 pc	00269cb5	/system/lib/libhwui.so (_ZN7android10uirenderer7Texture13deleteTextureEv+12)					
13	#05 pc	0009ef4f	/system/lib/libhwui.so (_ZN7android10uirenderer7GLayerD2Ev+34)					

典型的Java堆栈

```
java.lang.OutOfMemoryError: pthread_create (1040KB stack) failed: Try again

at java.lang.Thread.nativeCreate(Thread.java)

at java.lang.Thread.start(Thread.java:883)

at java.util.concurrent.ThreadPoolExecutor.addWorker(ThreadPoolExecutor.java:975)

at java.util.concurrent.ThreadPoolExecutor.execute(ThreadPoolExecutor.java:1382)
```

典型的log如下

3075	02-12 19:49:00.174	25427 26990	WARNING	Adreno-GSL	<sharedmem_gpuobj_alloc:2461>: sharedmem_gpumem_alloc: mmap failed errno 12 Out of memory
3076	02-12 19:49:00.175	25427 26990	ERROR	Adreno-GSL	<gsl_memory_alloc_pure:2236>: GSL MEM ERROR: kgs_l_sharedmem_alloc ioctl failed.
3077	02-12 19:49:00.176	25427 26990	WARNING	Adreno-GSL	<sharedmem_gpuobj_alloc:2461>: sharedmem_gpumem_alloc: mmap failed errno 12 Out of memory
3078	02-12 19:49:00.177	25427 26990	ERROR	Adreno-GSL	<gsl_memory_alloc_pure:2236>: GSL MEM ERROR: kgs_l_sharedmem_alloc ioctl failed.
3079	02-12 19:49:00.177	25427 26990	WARNING	Adreno-GSL	<sharedmem_gpuobj_alloc:2461>: sharedmem_gpumem_alloc: mmap failed errno 12 Out of memory
3080	02-12 19:49:00.179	25427 26990	ERROR	Adreno-GSL	<gsl_memory_alloc_pure:2236>: GSL MEM ERROR: kgs_l_sharedmem_alloc ioctl failed.
3081	02-12 19:49:00.179	25427 26990	WARNING	Adreno-GSL	<sharedmem_gpuobj_alloc:2461>: sharedmem_gpumem_alloc: mmap failed errno 12 Out of memory
3082	02-12 19:49:00.181	25427 26990	ERROR	Adreno-GSL	<gsl_memory_alloc_pure:2236>: GSL MEM ERROR: kgs_l_sharedmem_alloc ioctl failed.
3107	02-12 19:56:09.139	25427 26990	ERROR	OpenGLRenderer	GL error: Out of memory!
3108	02-12 19:56:09.139	25427 26990	FATAL	OpenGLRenderer	glDeleteTextures error! GL_OUT_OF_MEMORY (0x505)
3116	02-12 19:56:10.784	25427 26990	ERROR	npth	Native crash was detected!

编写脚本，根据vma映射名字尝试Maps归类

编写脚本，根据vma映射名字尝试Maps归类

log	VmPeak	npth抓回线程数	map总大小
链接	3.8857 GB	276	3.87 GB

```
1 threadcount_with_stack_guard: 466
2 threadcount_with_signal_stack: 276
3 threadcount_with_bionic_tls: 276
4 threadcount_with_anon_stack: 0
```

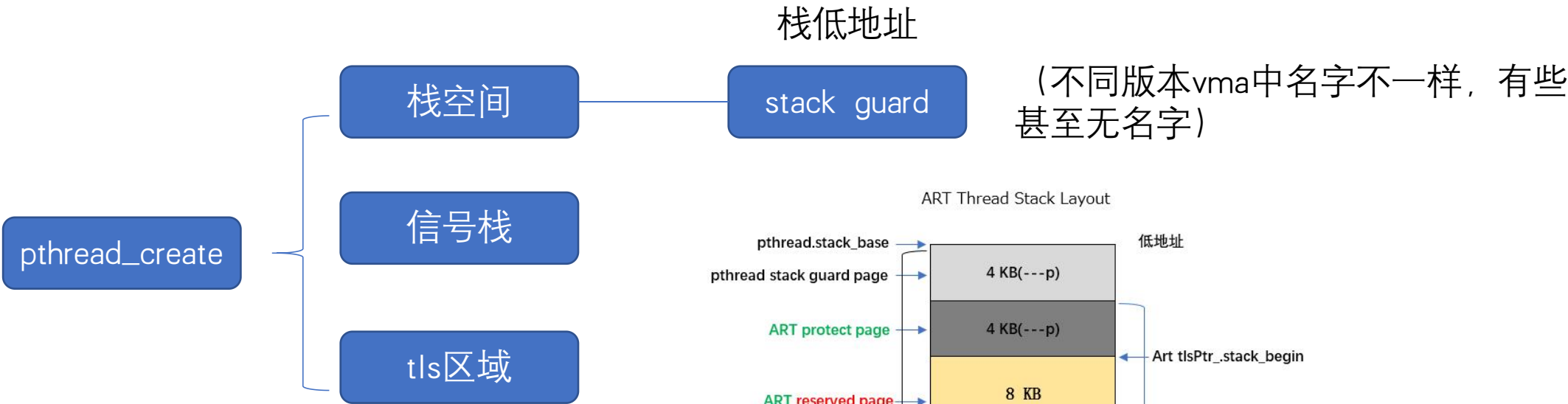
```
5 ===== maps1.txt =====
```

```
6      3958 MB      totals
7      1250 MB      dalvik
8      911 MB       malloc
9      545 MB       dmabuf
10     485 MB       thread stack
11     271 MB       java lib
12     184 MB       native file
13     117 MB       unknown
14     116 MB       ttf
```

从这个内存分布来看，主要的内存使用大头在前面几项：

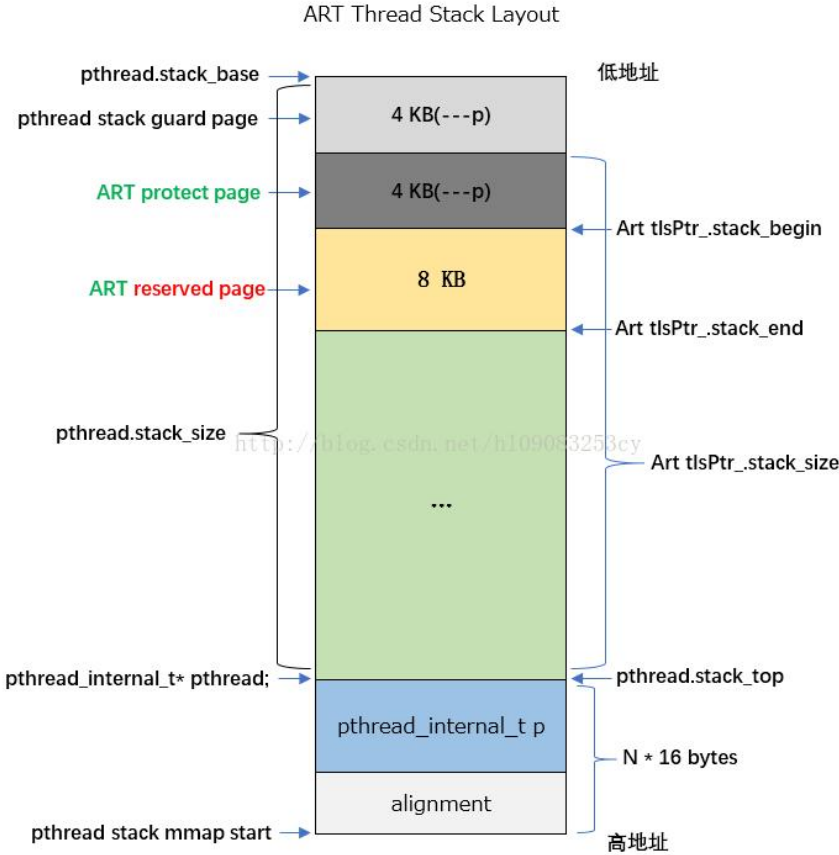
- dalvik
 - 这个使用空间最大，主要是虚拟机使用，当前可以确认无优化空间
- malloc
 - 这里统计的是allocator 分配的内存，这里 911M的使用显然不合理，应存在内存泄漏，需要分析解决，本文里重点不在这
- dmabuf
 - dmabuf占用的空间，目前还不完全明确谁使用的
- thread stack
 - 线程栈内存 485M，使用内存也较多

7.1源码分析线程创建



很明显: stack_guard有异常!

```
1 threadcount_with_stack_guard: 466
2 threadcount_with_signal_stack: 276
3 threadcount_with_bionic_tls: 276
4 threadcount_with_anon_stack: 0
5 ===== maps1.txt =====
```



pthread_exit源码求证

分析pthread_exit源码求证

```
62 void pthread_exit(void* return_value) {
63     // Call dtors for thread_local objects first.
64     __cxa_thread_finalize();
65
66     pthread_internal_t* thread = __get_thread();
67     thread->return_value = return_value;
68
69     // Call the cleanup handlers.
70     while (thread->cleanup_stack) {
71         pthread_cleanup_t* c = thread->cleanup_stack;
72         thread->cleanup_stack = c->__cleanup_prev;
73         c->__cleanup_routine(c->__cleanup_arg);
74     }
75
76     // Call the TLS destructors. It is important to do that before removing this
77     // thread from the global list. This will ensure that if someone else deletes
78     // a TLS key, the corresponding value will be set to NULL in this thread's TLS
79     // space (see pthread_key_delete).
80     pthread_key_clean_all();
81
82     if (thread->alternate_signal_stack != NULL) {
83         // Tell the kernel to stop using the alternate signal stack.
84         stack_t ss;
85         ss.ss_sp = NULL;
86         ss.ss_flags = SS_DISABLE;
87         sigaltstack(&ss, NULL);
88
89         // Free it.
90         munmap(thread->alternate_signal_stack, SIGNAL_STACK_SIZE);
91         thread->alternate_signal_stack = NULL;
92     }
```

释放信号栈

只有DETACHED才会走到下面的逻辑

```
99 if (old_state == THREAD_DETACHED) {
100     // The thread is detached, no one will use pthread_internal_t after pthread_exit.
101     // So we can free mapped space, which includes pthread_internal_t and thread stack.
102     // First make sure that the kernel does not try to clear the tid field
103     // because we'll have freed the memory before the thread actually exits.
104     __set_tid_address(NULL);
105
106     // pthread_internal_t is freed below with stack, not here.
107     __pthread_internal_remove(thread);
108
109     if (thread->mmap_size != 0) {
110         // We need to free mapped space for detached threads when they exit.
111         // That's not something we can do in C.
112
113         // We don't want to take a signal after we've unmapped the stack.
114         // That's one last thing we can handle in C.
115         sigset_t mask;
116         sigfillset(&mask);
117         sigprocmask(SIG_SETMASK, &mask, NULL);
118
119         __exit_with_stack_tear_down(thread->attr.stack_base, thread->mmap_size);
120     }
121 }
122
```

```
29 #include <private/bionic_asm.h>
30
31 // void __exit_with_stack_tear_down(void* stackBase, size_t stackSize)
32 ENTRY_PRIVATE(__exit_with_stack_tear_down)
33     mov w8, __NR_munmap
34     svc #0
35     // If munmap failed, we ignore the failure and exit anyway.
36
37     mov x0, #0
38     mov w8, __NR_exit
39     svc #0
40     // The exit syscall does not return.
41 END(__exit_with_stack_tear_down)
42
```

通过系统调用释放线程栈

找到业务泄露点

求证，多个实例泄露

Event详情

原始日志 / 3935

使用时，请遵守《字节跳动用户个人数据使用行为规范》

StackLeakSoName

Device ID	客户端发生时间	数据上报时间	类型	Message	系统版本	小版本	大版本
52656880903	2020-05-15 14:55:34+08:00	2020-05-15 14:55:37+08:00	stackleak	stackleak	7.1.2	11009900	11.0.0
设备机型	分辨率	渠道	ROM	Build	NPTH版本	网络	电量
xiaomi Redmi Note 5A(5.1568507)	1280x720	xiaomi	miui_V11_V11.0.1.0.NDKCNXM	5f51528_20200507_739734d2-9058-11ea-8899-02420a000052	3.1.2-rc.10	wifi	---
CPU架构	使用时长	是否后台	当前线程	当前进程			
arm64-v8a, armeabi-v7a, armeabi	---	否	StackLeakChecker Thread	com.ss.android.ugc.aweme			

堆栈详情

Retrace

跟踪数据

自定义数据

符号表/mapping

stack_leak_info

leak countoffsetleak so

0 java.lang.Throwable: [153] stacks leaked: [2272d] 05091000-c50dc000 r-xp [libttmain.so (51fc50d17e91170f81df05c03f9d3744df55904)]

Event详情

原始日志 / 11226

使用时，请遵守《字节跳动用户个人数据使用行为规范》

StackLeakSoName

Device ID	客户端发生时间	数据上报时间	类型	Message	系统版本	小版本	大版本
39729140127	2020-05-15 14:56:07+08:00	2020-05-15 14:56:38+08:00	stackleak	stackleak	6.0.1	11009900	11.0.0
设备机型	分辨率	渠道	ROM	Build	NPTH版本	网络	电量
vivo Y53L(5.2732515)	960x540	vivo	Funtouch OS_3.0_PD1628_A_1.16.31	5f51528_20200507_739734d2-9058-11ea-8899-02420a000052	3.1.2-rc.10	wifi	---
CPU架构	使用时长	是否后台	当前线程	当前进程			
armeabi-v7a, armeabi	---	否	StackLeakChecker Thread	com.ss.android.ugc.aweme:push			

堆栈详情

Retrace

跟踪数据

自定义数据

符号表/mapping

stack_leak_info

leak countoffsetso build id

0 java.lang.Throwable: [104] stacks leaked: [191d] 9064b000-9064f000 r-xp [libdaemon.so (4275ffac2bee51232c882280ed0ca7bb642f903d)]

修复逻辑

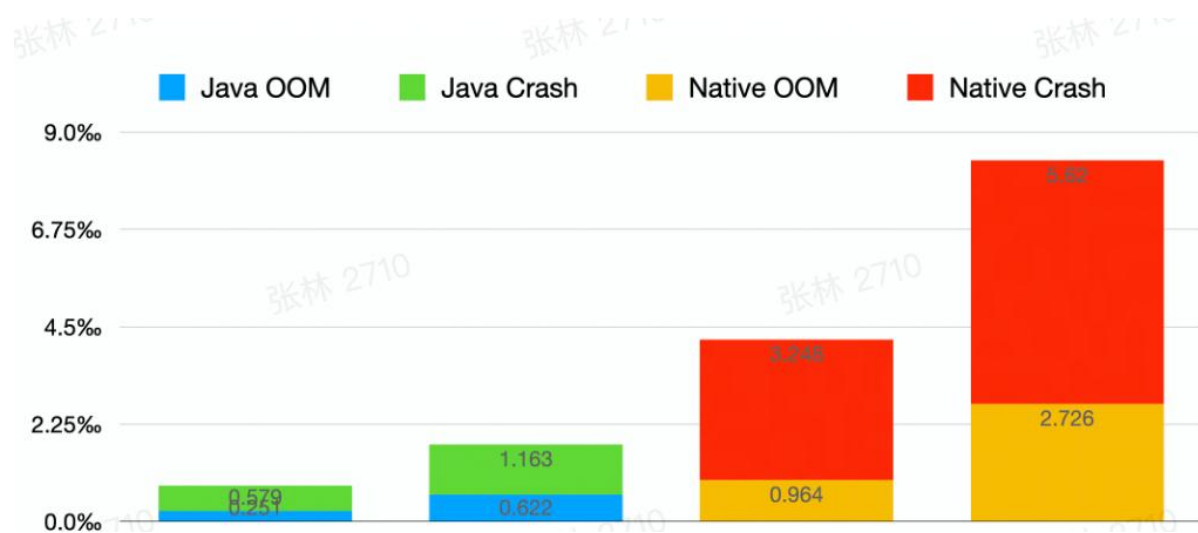
1. 通过pthread_create创建线程时，设置 detached状态，参考如下代码：

```
1 pthread_t file_observer_thread;
2
3 pthread_attr_t attr;
4 pthread_attr_init(&attr);
5 pthread_attr_setdetachstate (&attr, PTHREAD_CREATE_DETACHED);
6
7 int ret_file_observer_thread = pthread_create(&file_observer_thread,
8 &attr, (void *) start_file_observer, NULL);
```

2. 在函数开始执行时把线程设置 detached状态，参考如下代码：

```
1 void *start_file_observer(void *args) {
2 pthread_detach(pthread_self());
3 }
```

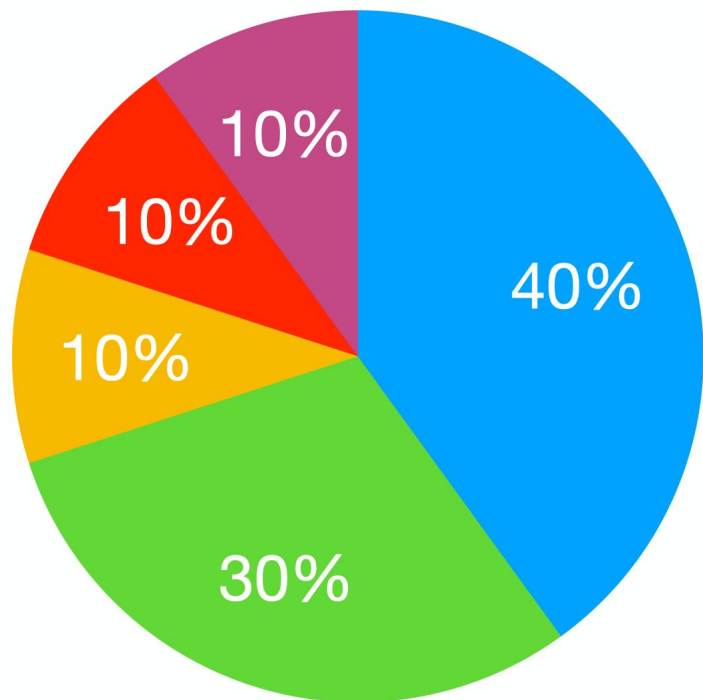
经典Case4: OOM治理



- Java OOM指标上涨150%
- Native OOM指标上涨180%
- 线上Java OOM激增导致发版delay

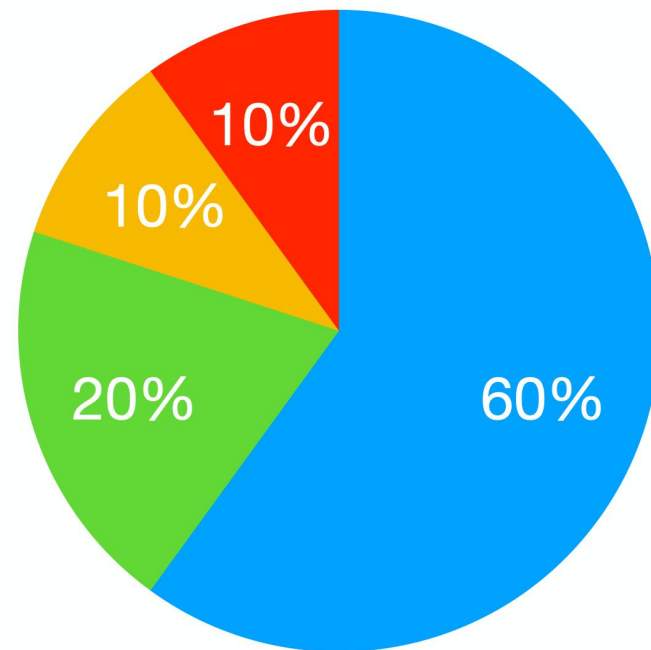
Top10 分析

Java



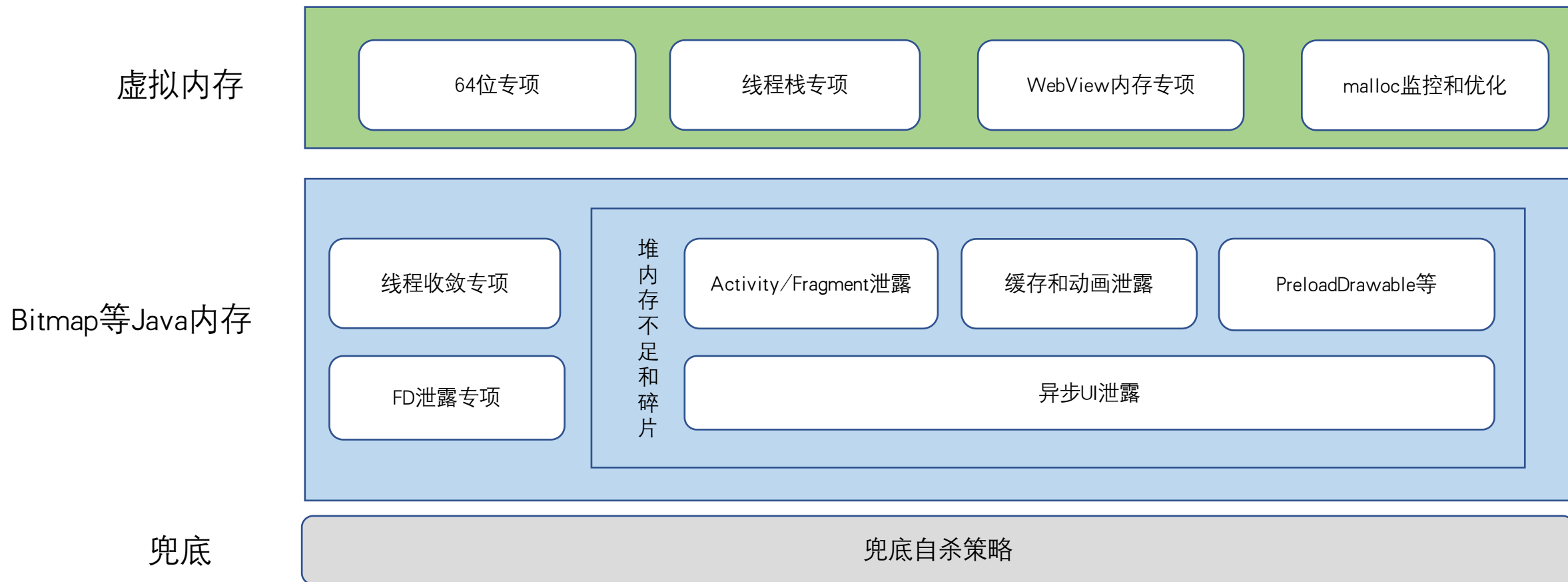
- 虚拟内存不足
- Bitmap创建失败
- 线程超500
- Java堆内存不足
- Java堆内存碎片

Native



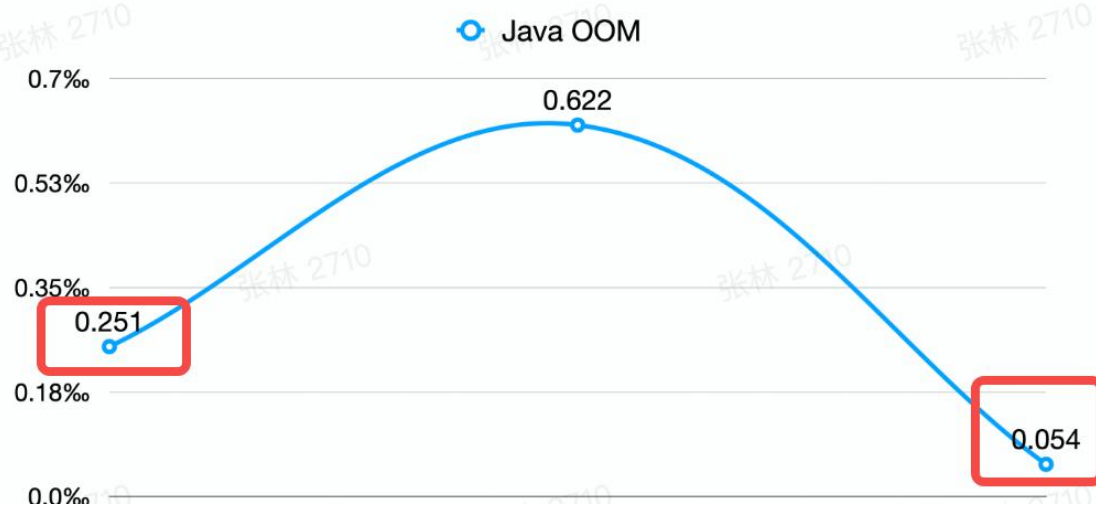
- 虚拟内存不足
- 线程超300
- 线程超400
- FD泄漏

OOM治理专项

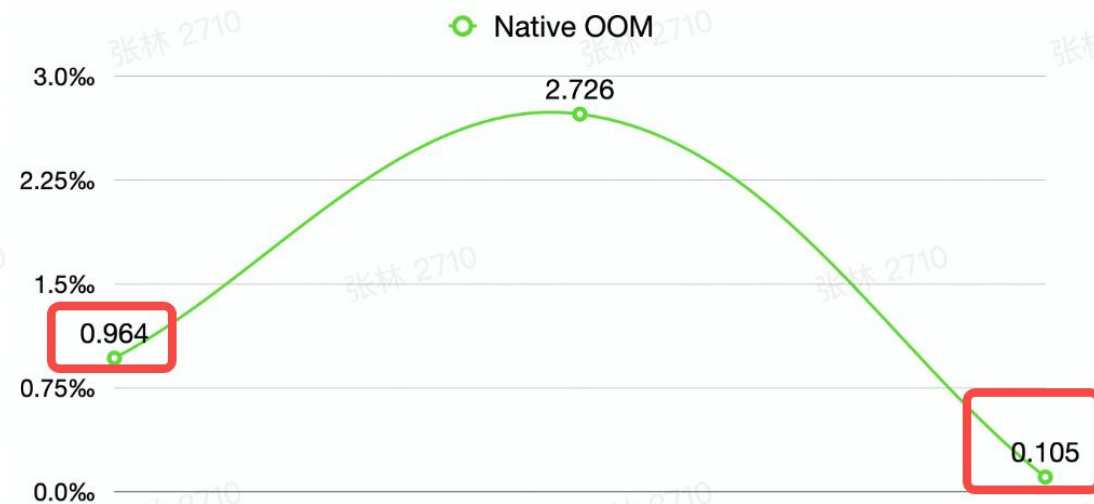


OOM治理专项

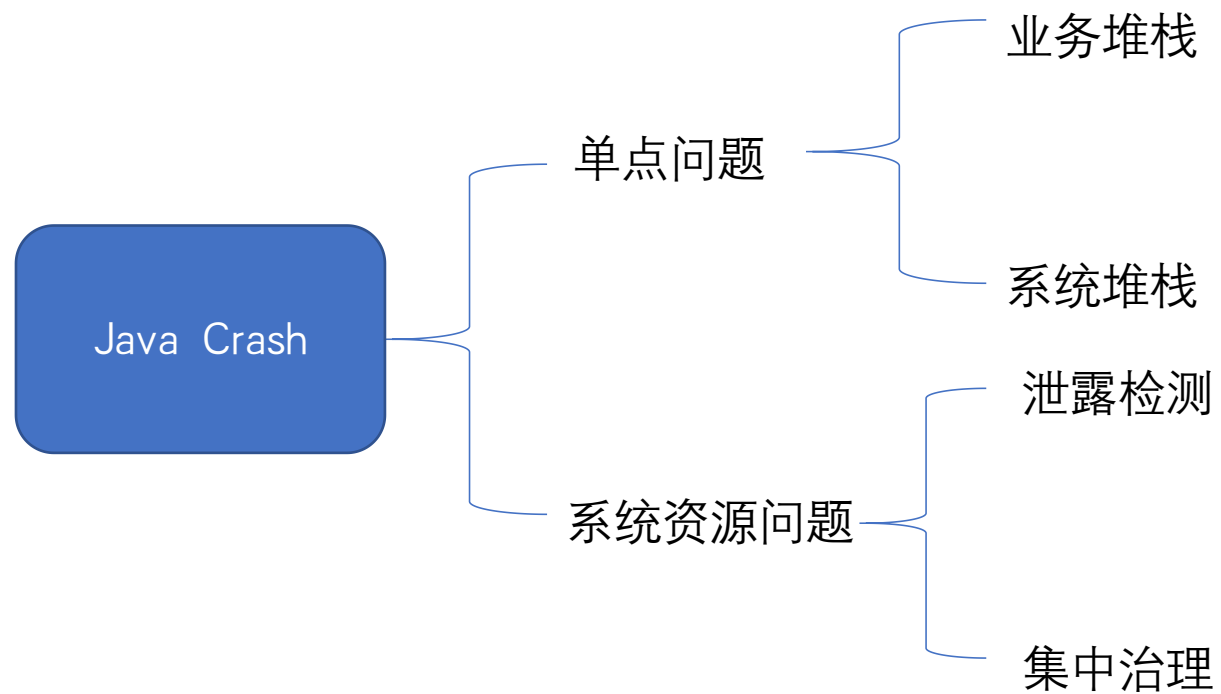
Java OOM优化效果



Native OOM优化效果



疑难攻坚-Java Crash



系统单点问题

排查思路

logcat业务和系统日志



更多函数信息排查（切面）

字节码hook框架：lancet

动态trace信息：插桩+下发配置

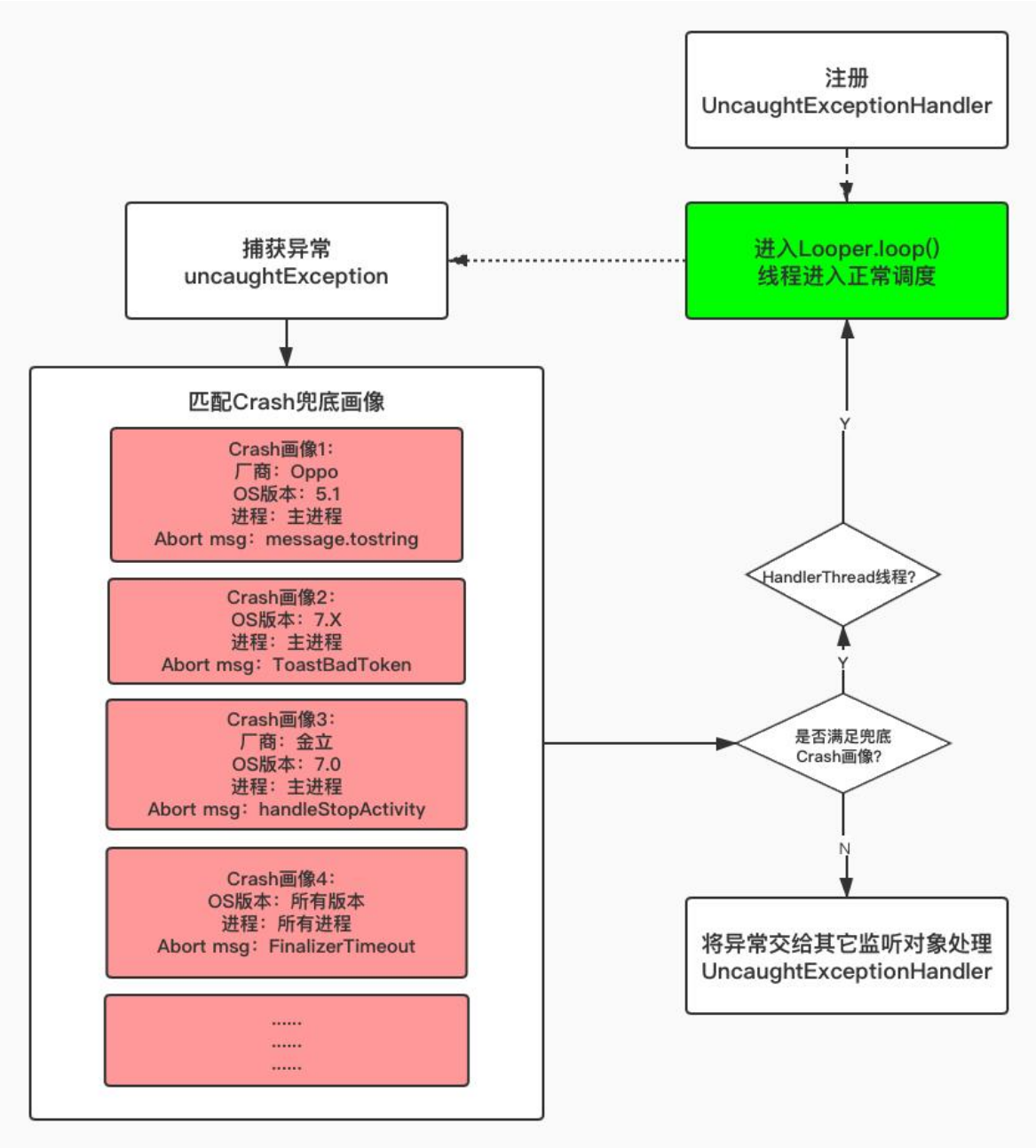
离线java hook框架：xposed、frida

解决

更改代码

兜底：Loop兜底框架

Loop兜底框架



Loop兜底案例

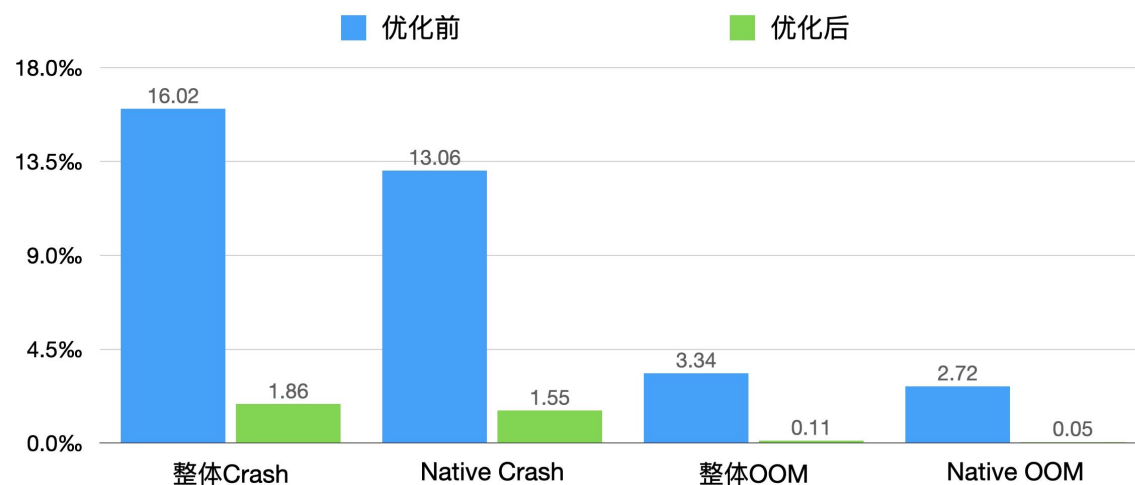
```
java.lang.ArrayIndexOutOfBoundsException: Array index out of range: 8
at android.util.ArrayMap.getValueAt(ArrayMap.java:523)
at android.app.ActivityThread.collectComponentCallbacks(ActivityThread.java:5627)
at android.app.ActivityThread.handleTrimMemory(ActivityThread.java:6362)
at android.app.ActivityThread.access$1200(ActivityThread.java:246)
at android.app.ActivityThread$ApplicationThread.lambda$scheduleTrimMemory$0(ActivityThread.java:1642)
at android.app.-$$Lambda$ActivityThread$ApplicationThread$tUGFX7CUhzB4Pg5wFd5yeqOnu38.accept(Unknown Source:8) {
at com.android.internal.util.function.pooled.PooledLambdaImpl.doInvoke(PooledLambdaImpl.java:278)
at com.android.internal.util.function.pooled.PooledLambdaImpl.invoke(PooledLambdaImpl.java:201)
at com.android.internal.util.function.pooled.OmniFunction.run(OmniFunction.java:97)
at android.view.Choreographer$CallbackRecord.run(Choreographer.java:1077)
at android.view.Choreographer.doCallbacks(Choreographer.java:897)
at android.view.Choreographer.doFrame(Choreographer.java:828)
at android.view.Choreographer$FrameDisplayEventReceiver.run(Choreographer.java:1062)
at android.os.Handler.handleCallback(Handler.java:938)
at android.os.Handler.dispatchMessage(Handler.java:99)
at android.os.Looper.loop(Looper.java:233)
at android.app.ActivityThread.main(ActivityThread.java:7892)
at java.lang.reflect.Method.invoke(Native Method)
at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:656)
at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:967)
```

配置如下：

```
{
  "exceptionMessage": "length=0; index=1",
  "exceptionType": "java.lang.ArrayIndexOutOfBoundsException",
  "stackClassName": "android.app.ActivityThread",
  "stackMethodName": "collectComponentCallbacks"
}
```

疑难问题治理效果

- **Native**: DvmLinearAlloc, 虚拟机Suspend超时, 三星机型ArtField, Leaked 4 GPU objects 等
- **OOM**: NativeBitmap, 线程栈泄漏, 64位等
- **Java**: ActivityBadToken, DeadSystem
- **SysOptimizer**: 通用质量优化框架, 集成方案 39+, 接入产品62+, 稳定性指标优化40%+



04

发版质量专项

背景

数量巨大

灰度从上万个问题list筛选

人工操作

人工筛选分配

反复确认是否修复

人工跟进问题

疑难问题不知道如何修复

没有标准

很难直观看出当前版本质量如何

不清楚版本是否可以全量

频繁跳板和delay

思路

1. walle根据堆栈和其他信息自动分配
2. 根据策略，自动筛选必解问题
3. 必解问题自动播报，专家系统辅助分析
4. 机器人自动推进问题修复
5. 各版本曲线对比
6. 版本报告

质量视图落地效果

Java Crash	Startup Crash	Native Crash	ANR	Leak	Slow Function	Memory Degradation	All	unsolved	P0	P1
New Created	Unsettled	Processing	Long-term	Solved	No.	Reopen	Unassigned	Jump To Slardar		
Anomaly Details	Managers	Status	Count	Users	Dist Detail	关联				
SIGTRAP libmonochrome.so libc.so __unregister_atfork	wangwei.bw	Solved	2457	1183 Top344 -> Top3	2022009030- 2022702050 App2022702010 Android11.0 googleplay	跟进群				
SIGTRAP libmonochrome.so libc.so pthread_attr_init libc.so pthread_attr_init	wangwei.bw	Solved	355	141 Top397 -> Top8	2022009020- 2022702050 Android11.0 googleplay	跟进群				

年度平均0跳板，0delay

▲BM需要推动严重问题的Owner第一优先级解决，避免版本Delay。

Java启动崩溃
(当前无必解问题)

Java崩溃
1. 【P1】 [Top648->Top19] DelegateRunnable.java [状态:Long-term]

内存泄漏
(当前无必解问题)

ANR
(当前无必解问题)

- Native崩溃
- 1. 【P1】 [Top119->Top13] libc++.so std::__1... [状态:Long-term]
 - 2. 【P1】 [Top212->Top25] libc++.so std::__1... [状态:Unsettled]
 - 3. 【P1】 [Top102->Top29] libhwui.so android... [状态:Unsettled]

05

线上容灾专项

背景

线上报警频繁，指标居高不下

20+线上变更平台无客户端监控

json，正则等低级错误频发

纯人工响应

正面分析时间巨长

周报警几十个

无熔断保护功能

启动崩溃，实验无法及时下发

思路

思路

事故发生时:

线上变更归因

事故发生前:

精准熔断

兜底:

智能保护

容灾阶段-自动归因

分钟级自动归因

请求匹配

100个DID崩溃前path请求分布

请求占比 72.0%:  /stats/ (该path暂未找到对应psm)

请求占比 60.0%:  /feedback/ (该path暂未找到对应psm)

6182-线上报警 DID命中实验情况

格式说明: 【实验组命中率/实验组命中数/实验命中数/did数】 【放量】 实验及链接

排序说明: 按照 【实验组命中数】 的平方 / 【实验命中数】 排序

【100.000%/183/183/183】 【放量20.0%】 【品质-资源】 直播Gecko清理实验

【96.721%/177/183/183】 【放量100.0%】 RTA-HUAWEI非商店-抖音卸载召回-高线高知-全实验层

【95.082%/174/183/183】 【放量100.0%】 抖音拉新首日屏蔽实验

【95.082%/174/183/183】 【放量100.0%】 抖音_电商端外拉活_m9m10_v2

【94.536%/173/183/183】 【放量100.0%】 抖音拉新蚕食实验PG渠道

【94.536%/173/183/183】 【放量100.0%】 商家惊喜券-抖音主端-1104

【libra页面查询工具】 :

[libra用户命中查询接口链接](#)

did自动匹配

容灾阶段-手动归因

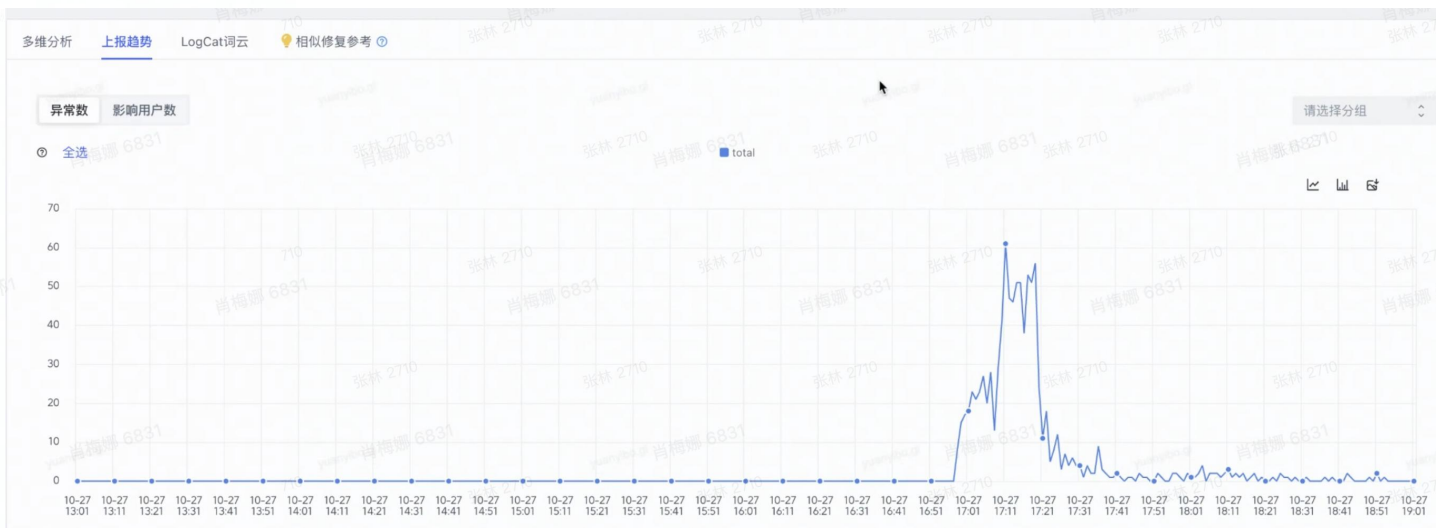
按归因时间维度做可视化聚合算法



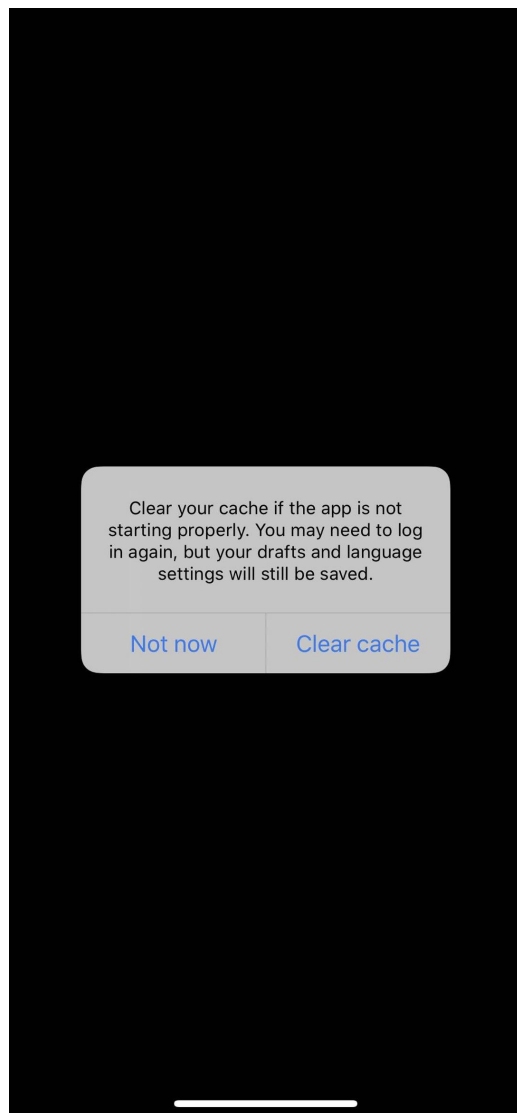
容灾阶段-精准熔断

- 2022-10-26 12:16:50 datarocks
实验分组v2的测试用户: 移除 7140532346333759022
- 2022-10-27 02:23:27 [REDACTED]
实验分组v2的测试用户:
将实验分组v2的测试用户修改为7156122089295595010
- 2022-10-27 02:31:13 Libra
精准熔断平台检测中, 放量前30min只增加4.0%
- 2022-10-27 02:31:13 [REDACTED]
启动实验, 从测试状态进入运行状态
- 2022-10-27 03:00:46 Libra
精准熔断平台检测正常, 结束精准熔断
- 2022-10-27 03:01:04 IESQA-RIG
精准熔断平台检测异常, 触发熔断, 实验已暂停
- 2022-10-27 03:01:04 abtest_team_only
暂停实验: 将状态从运行中修改为已暂停

精准定位风险, 无人值守, 自动回滚



容灾阶段-智能保护



崩溃守护



容灾专项-效果

线上变更归因

50+ | 60%

线上事故

成功率

- 接入17线上配置变更平台
- 每小时变更数量 100+
- 分钟级别的自动归因，事故归因时间从小时级下降到分钟级

精准熔断

9 | 30

P0事故

分钟

- 2022年P2+事故下降100%
- 召回和准确率 60%+
- 30分钟无人值守止损
- 支持11类变更平台熔断
- 接入抖音、头条等8个产品

智能保护

14 | 80%

启动崩溃

成功率

- 启动保护：检测启动崩溃，并回退到上一个正确的配置
- 安全气囊：检测Json解析、正则等通用异常，触发兜底
- 避免1例 5千万用户级别事故

06

质量左移专项

质量左移专项

线下问题发现&代码准入

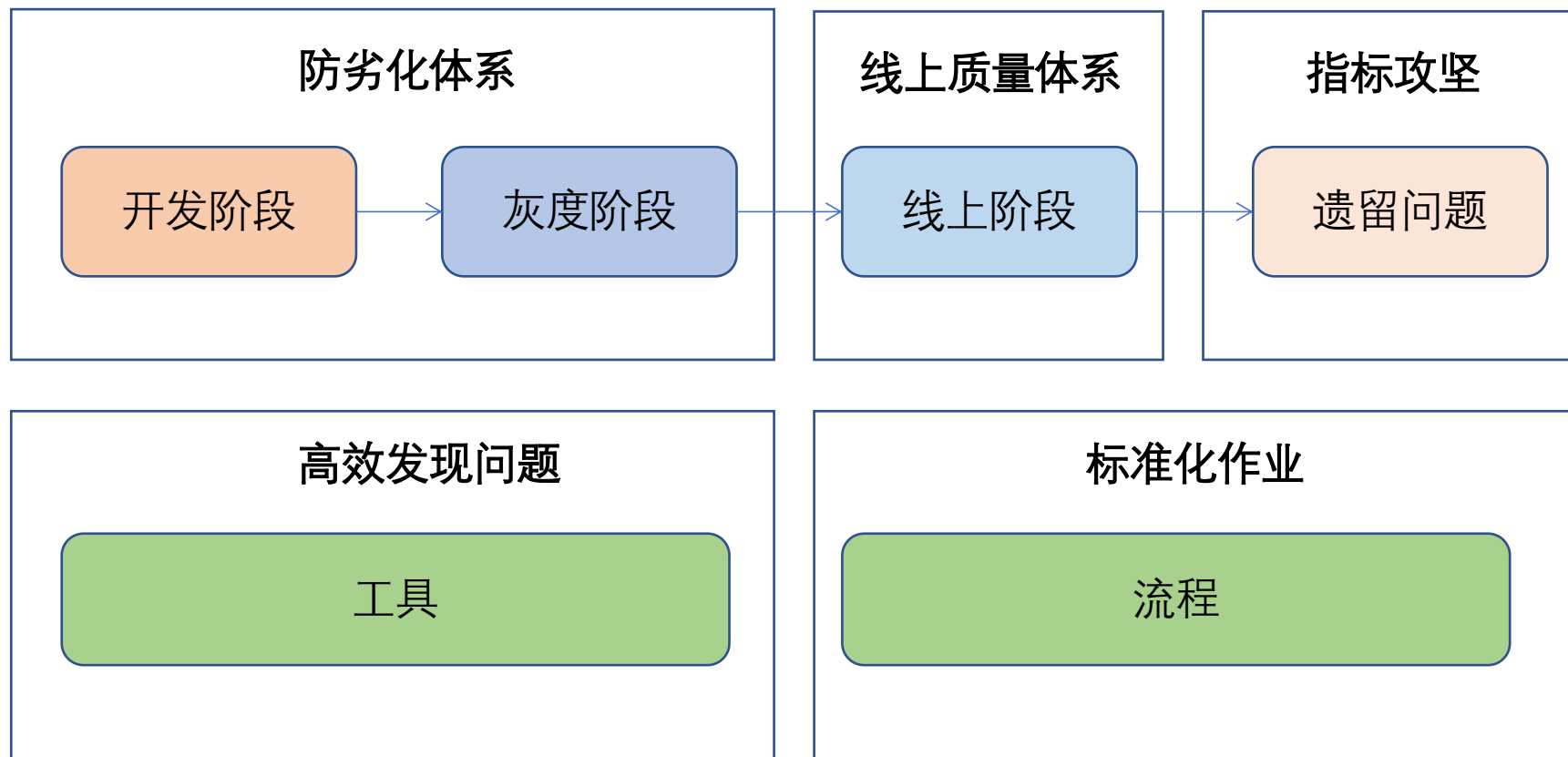


灰度必解问题下降30%

07

总结与展望

全景图



总结

攻坚

hook

汇编

逆向

改内存

改指令

兜底

修改业务

...

版本

版本自动化体系

问题自动筛选

自动推进

自动播报

自动报告

版本防劣化体系

线上

线上变更归因

精准熔断

智能保护

左移

各种手段并行，多角度发现问题

展望：Rig平台，开箱即用的一站式质量平台

← → ↻ rig-global.bytedance.net/home

 RIG 质量平台

敬请期待

RIG FOR QUALITY

开箱即用的一站式客户端质量平台

开始使用

介绍文档





THANKS.